

2026 Security Assessment Report Prepared For



Report Issued: 17/1/2026



Aviso de confidencialidad

Este documento contiene información sensible relacionada con la seguridad de una aplicación web. El contenido de este informe es estrictamente confidencial y está destinado únicamente a las personas autorizadas para su revisión en el contexto de la presente auditoría.

Queda prohibida la reproducción, distribución o divulgación total o parcial de este documento sin la autorización expresa del autor.

El acceso no autorizado a este documento o el uso indebido de la información contenida en él puede dar lugar a responsabilidades legales.

Exención de responsabilidad

Las vulnerabilidades identificadas y las técnicas de explotación aplicadas forman parte de un entorno intencionadamente vulnerable.

El autor no se hace responsable del uso indebido de la información contenida en este informe ni de los daños que pudieran derivarse de la aplicación de estas técnicas en sistemas reales sin la debida autorización.

Los resultados y conclusiones presentados en este documento reflejan el estado de la aplicación en el momento de la auditoría y no garantizan la ausencia de vulnerabilidades adicionales.

Tabla de contenido

Índice

1. Resumen ejecutivo	5
1.1 Resumen del proceso realizado	5
1.2 Vulnerabilidades destacadas	5
1.3 Conclusiones	6
1.4 Recomendaciones	6
 2. Visión general de la auditoría	 7
2.1 Metodología de la auditoría	7
2.1.1 Metodología de pruebas	9
2.2 Herramientas utilizadas	9
 3. Fases de la auditoría	 10
3.1 Reconocimiento activo	10
3.2 Identificación de tecnologías	10
3.3 Superficie de ataque	11
 4. Vulnerabilidad A3 – Inyección SQL (SQL Injection)	 11
4.1 Descripción de la vulnerabilidad	11
4.2 Vector de ataque identificado	12
4.3 Explotación de la vulnerabilidad	12
4.4 Evidencias	12
4.5 Impacto	14
4.6 Nivel de riesgo	14
4.7 Recomendaciones de mitigación	14
 5. Vulnerabilidad A3 – Cross-Site Scripting (XSS)	 15
5.1 Descripción de la vulnerabilidad	15
5.2 Vector de ataque identificado	15
5.3 Explotación de la vulnerabilidad	15
5.4 Evidencias	15
5.5 Impacto	16
5.6 Nivel de riesgo	16
5.7 Recomendaciones de mitigación	16

6. Vulnerabilidad A5 – Security Misconfiguration	17
6.1 Descripción de la vulnerabilidad	17
6.2 Vector de ataque identificado	17
6.3 Explotación / Identificación	17
6.4 Evidencias	18
6.5 Impacto	19
6.6 Nivel de riesgo	20
6.7 Recomendaciones de mitigación	20
 7. Vulnerabilidad A6 – Vulnerable & Outdated Components	 20
7.1 Descripción de la vulnerabilidad	20
7.2 Identificación en el entorno analizado	20
7.3 Explotación / Evidencia	21
7.4 Impacto	21
7.5 Nivel de riesgo	21
7.6 Recomendaciones de mitigación	21
 8. Vulnerabilidad A7 – Identity & Authentication Failure	 22
8.1 Descripción de la vulnerabilidad	22
8.2 Identificación en el entorno analizado	22
8.3 Análisis del problema	22
8.4 Evidencias	24
8.5 Impacto	25
8.6 Nivel de riesgo	25
8.7 Recomendaciones de mitigación	25
 9. Conclusiones finales	 25

1. Resumen ejecutivo

1.1 Resumen del proceso realizado

Se ha llevado a cabo una auditoría de seguridad sobre la aplicación web **WebGoat 8.1.0**, con el objetivo de identificar y explotar vulnerabilidades comunes en aplicaciones web, siguiendo las directrices del **OWASP Top 10**.

Durante el proceso de auditoría se realizaron distintas fases, incluyendo reconocimiento inicial, análisis de la superficie de ataque, explotación de vulnerabilidades y evaluación del impacto de las mismas. Para ello se combinaron técnicas manuales con el uso de herramientas especializadas de seguridad.

1.2 Vulnerabilidades destacadas

Como resultado del análisis realizado, se identificaron y explotaron diversas vulnerabilidades relevantes, entre las que destacan:

- **Inyección SQL (SQL Injection)**, que permite la manipulación de consultas a la base de datos y el acceso no autorizado a información sensible.
- **Cross-Site Scripting (XSS)**, posibilitando la ejecución de código JavaScript malicioso en el navegador de otros usuarios.
- **Configuraciones de seguridad incorrectas (Security Misconfiguration)**, que facilitan el acceso a funcionalidades o información que deberían estar restringidas.
- **Uso de componentes vulnerables u obsoletos**, aumentando el riesgo de explotación mediante vulnerabilidades conocidas.
- **Fallos en la gestión de identidades y autenticación**, comprometiendo la seguridad de las credenciales de los usuarios.

Estas vulnerabilidades han sido clasificadas en función de su impacto potencial y facilidad de explotación.

1.3 Conclusiones

La auditoría pone de manifiesto que la aplicación WebGoat presenta múltiples vulnerabilidades críticas desde el punto de vista de la seguridad.

No obstante, en un entorno real de producción, la presencia de este tipo de vulnerabilidades podría derivar en accesos no autorizados, exposición de datos sensibles, manipulación de información y compromiso completo del sistema.

Este ejercicio permite comprender la importancia de aplicar buenas prácticas de desarrollo seguro y realizar auditorías periódicas para reducir la superficie de ataque de las aplicaciones web.

1.4 Recomendaciones

Se recomienda implementar las siguientes medidas de seguridad:

- Validar y sanitizar adecuadamente todas las entradas de usuario.
- Utilizar consultas preparadas para prevenir ataques de inyección SQL.
- Implementar mecanismos de protección frente a XSS, como el uso de codificación de salida y cabeceras de seguridad.
- Mantener actualizados todos los componentes y dependencias de la aplicación.
- Aplicar configuraciones de seguridad adecuadas y principios de mínimo privilegio.
- Realizar auditorías de seguridad de forma periódica.

La adopción de estas medidas contribuirá significativamente a mejorar la postura de seguridad de aplicaciones web en entornos reales.

La implementación de estas medidas mitigaría las principales vías de ataque detectadas durante la auditoría.

2. Visión general de la auditoría

La auditoría de seguridad realizada sobre la aplicación web WebGoat 8.1.0 ha permitido obtener una visión global del estado de seguridad de la aplicación desde un enfoque de alto nivel. El análisis reveló varias vulnerabilidades comunes en aplicaciones web, principalmente relacionadas con fallos de validación de entradas, configuraciones de seguridad incorrectas, uso de componentes vulnerables y deficiencias en los mecanismos de autenticación y control de acceso.

El nivel general de riesgo de la aplicación puede considerarse elevado, ya que varias de las vulnerabilidades detectadas permiten la explotación directa sin necesidad de autenticación previa y podrían derivar en la exposición de información sensible o en la ejecución de acciones no autorizadas. Los resultados obtenidos reflejan escenarios realistas que pueden encontrarse en aplicaciones web reales que no siguen buenas prácticas de desarrollo seguro.

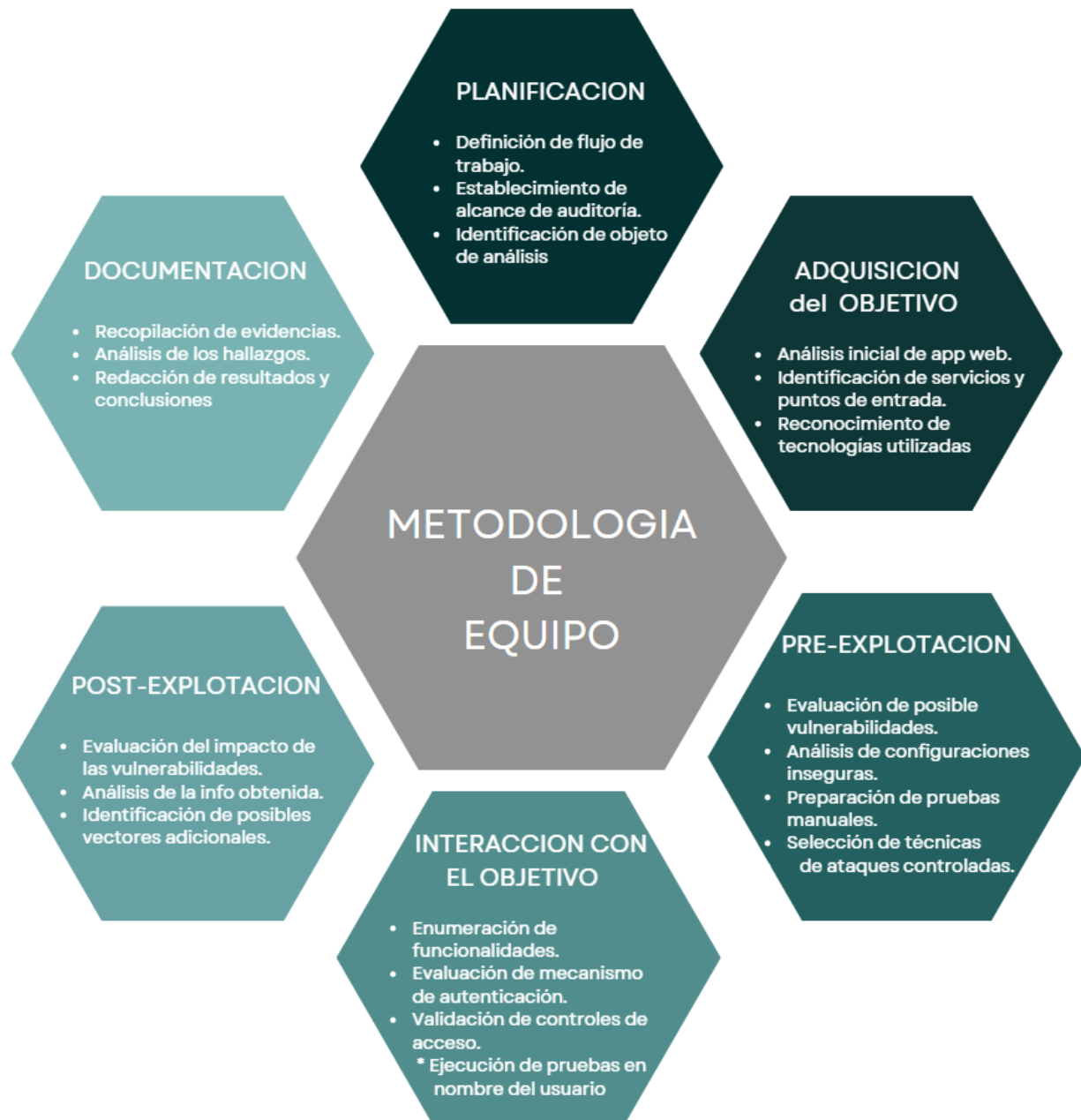
El alcance de esta auditoría se ha limitado a la instancia local de WebGoat desplegada en un contenedor Docker, sin incluir sistemas externos, otros entornos o revisión de código fuente. Esta visión general sirve como introducción a los hallazgos identificados, los cuales se detallan en las secciones posteriores del informe.

2.1 Metodología de la auditoría

El objetivo ha sido identificar y explotar vulnerabilidades comunes en aplicaciones web, analizando sus posibles consecuencias y sugiriendo soluciones adecuadas.

El proceso de auditoría se ha dividido en varias fases claramente diferenciadas, que se describen a continuación.

A continuación, se muestra un esquema que resume las fases clave de la auditoría, desde la planificación inicial hasta la documentación de hallazgos:



[Elaboración propia]

2.1.1 Metodología de Pruebas

La metodología de pruebas empleada durante esta auditoría se estructuró en varias fases con el objetivo de analizar de forma ordenada las vulnerabilidades presentes en la aplicación web evaluada.

En una primera fase de reconocimiento, se recopiló información general sobre la aplicación WebGoat y su entorno de ejecución, identificando tecnologías utilizadas y puntos de entrada relevantes para las pruebas posteriores.

A continuación, se llevó a cabo una fase de análisis y evaluación dirigida, en la que se revisaron los distintos apartados solicitados en el Trabajo, centrándose en vulnerabilidades relacionadas con inyección, configuraciones inseguras, uso de componentes vulnerables y fallos en la autenticación.

Finalmente, se realizó una fase de validación y pruebas de explotación realizadas en un entorno aislado identificadas.

Durante esta fase se recopilaron evidencias que demuestran la existencia y el impacto de las vulnerabilidades analizadas.

2.2 Herramientas utilizadas

Durante la realización de la auditoría se utilizaron diversas herramientas con el objetivo de analizar el comportamiento de la aplicación y validar las vulnerabilidades identificadas. Las principales herramientas empleadas fueron las siguientes:

- **Nmap**: utilizado para la identificación de puertos abiertos y servicios expuestos durante la fase de reconocimiento.
- **Burp Suite**: empleado para el análisis del tráfico HTTP, inspección de peticiones y respuestas, y revisión de configuraciones de seguridad.
- **Navegador web y herramientas de desarrollo**: utilizados para la interacción directa con la aplicación, ejecución de pruebas manuales y análisis del comportamiento del lado cliente.
- **SQLMap**: herramienta considerada durante el análisis, aunque la explotación de inyecciones SQL se realizó de forma manual debido a las características del entorno WebGoat.
- **WhatWeb**: utilizado durante la fase de reconocimiento para la identificación de tecnologías web, frameworks, cookies de sesión y componentes expuestos por la aplicación.

3. Fases de la auditoría

El proceso de auditoría se estructuró en diferentes fases, comenzando por una etapa de reconocimiento orientada a identificar los servicios expuestos, tecnologías empleadas y posibles vectores de ataque. La información obtenida durante esta etapa permitió orientar eficazmente las fases posteriores de detección y explotación de vulnerabilidades.

3.1 Reconocimiento activo

En esta fase inicial se recopiló la máxima información posible sobre la aplicación web auditada, con el objetivo de identificar las tecnologías empleadas, los servicios expuestos y los posibles vectores de ataque. Durante la etapa de reconocimiento, se ejecutó un escaneo activo con Nmap sobre la instancia de WebGoat desplegada en un contenedor Docker alojado en una máquina virtual. Como resultado, se identificaron dos puertos TCP abiertos, 8080 y 9090, ambos vinculados a servicios HTTP gestionados por Apache Tomcat, lo que confirmó la exposición de la aplicación a través de este servidor.

Apache Tomcat es un servidor de aplicaciones ampliamente utilizado para el despliegue de aplicaciones web desarrolladas en Java, lo que sugiere que la aplicación auditada está basada en esta tecnología, si bien no se dispone de acceso al código fuente para confirmarlo de manera definitiva.

Asimismo, mediante las capacidades de detección del escaneo realizado, se identificó el sistema operativo subyacente como Linux, información coherente con el uso de contenedores Docker como entorno de despliegue.

3.2 Identificación de tecnologías

Se realizó un análisis complementario utilizando la herramienta WhatWeb, que permitió confirmar la presencia de componentes adicionales de la aplicación. El análisis reveló el uso de Java como tecnología backend, el framework Bootstrap en la capa de presentación, y la existencia de una página de autenticación accesible en la ruta /WebGoat/login.

Durante este análisis también se identificó la cookie de sesión JSESSIONID, configurada con el atributo HttpOnly, lo que limita el acceso a dicha cookie desde scripts del lado cliente y constituye una medida básica de mitigación frente a ataques de tipo Cross-Site Scripting (XSS). Asimismo, se observó una redirección inicial (HTTP 302) hacia la página de login, indicando que el acceso a determinadas funcionalidades de la aplicación requiere autenticación previa.

3.3 Superficie de ataque

También se efectuó un análisis manual desde el navegador web para evaluar el comportamiento de la aplicación desde el lado del cliente. Esto permitió identificar la superficie de ataque accesible para un usuario externo, así como las funcionalidades expuestas y los puntos de entrada iniciales.

```
(kali@kali)-[~]  
$ whatweb http://127.0.0.1:8080/WebGoat/  
http://127.0.0.1:8080/WebGoat/ [302 Found] Cookies[JSESSIONID], Country[RESERVED][ZZ], HttpOnly[JSESSIONID], IP[127.0.0.1], Java, RedirectLocation[http://127.0.0.1:8080/WebGoat/login]  
http://127.0.0.1:8080/WebGoat/login [200 OK] Bootstrap, Content-Language[en-US], Country[RESERVED][ZZ], HTML5, IP[127.0.0.1], PasswordField[password], Title[Login Page]
```

```
(kali@kali)-[~]  
$ nmap -sS -sV -O -Pn 127.0.0.1  
  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-01-10 06:07 -0500  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.000035s latency).  
Not shown: 998 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
8080/tcp  open  http    Apache Tomcat (language: en)  
9090/tcp  open  http    Apache Tomcat (language: en)  
Device type: general purpose  
Running: Linux 2.6.X|5.X  
OS CPE: cpe:/o:linux:linux_kernel:2.6.32 cpe:/o:linux:linux_kernel:5 cpe:/o:linux:linux_kernel:6  
OS details: Linux 2.6.32, Linux 5.0 - 6.2  
Network Distance: 0 hops  
  
OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 7.56 seconds
```

4. Vulnerabilidad A3 – Inyección SQL (SQL Injection)

4.1 Descripción de la vulnerabilidad

La inyección SQL es una vulnerabilidad que permite a un atacante manipular las consultas SQL que una aplicación envía a la base de datos, mediante la introducción de código malicioso a través de parámetros de entrada no validados correctamente.

En aplicaciones vulnerables, este tipo de ataque puede permitir el acceso no autorizado a información sensible, la modificación de datos o incluso el compromiso completo del sistema.

4.2 Vector de ataque identificado

Se identificó un punto de entrada crítico en un formulario web que interactúa directamente con la base de datos, donde los datos proporcionados por el usuario se incorporan a las consultas SQL sin aplicar validaciones adecuadas ni el uso de consultas preparadas.

La falta de validación habilitó la explotación directa mediante técnicas de inyección SQL, incluso con herramientas automatizadas como SQLMap.

4.3 Explotación de la vulnerabilidad

Continuando con la explotación manual de la vulnerabilidad de inyección SQL, se logró acceder a información adicional procedente de otras fuentes internas de la aplicación, más allá de los datos inicialmente expuestos.

A través de la manipulación de los parámetros de entrada y la ejecución de consultas orientadas a la extracción de información del sistema, la aplicación respondió con datos internos que no deberían estar disponibles para un usuario sin privilegios. Como se muestra en la Figura 1, fue posible acceder a múltiples registros que contienen identificadores de usuario, nombres de usuario, credenciales almacenadas y valores asociados a cookies de sesión. La obtención de esta información confirma que la vulnerabilidad permite no solo alterar la lógica de las consultas SQL, sino también acceder a datos sensibles almacenados en distintas estructuras internas de la aplicación, comprometiendo de forma directa la confidencialidad de la información.

4.4 Evidencias

Las evidencias recopiladas demuestran la explotación exitosa de la vulnerabilidad de inyección SQL y el acceso no autorizado a información interna de la aplicación.

Extracción de información interna adicional (usuarios, contraseñas simuladas y cookies) tras la explotación de SQL Injection.

✓

Employee Name:

from user_system_data; --

Authentication TAN:

TAN

Get department

You have succeeded! You successfully compromised the confidentiality of data by viewing internal information that you should not have access to. Well done!

USERID	USER_NAME	PASSWORD	COOKIE
101	jsnow	passwd1	
102	jdoe	passwd2	
103	jplane	passwd3	
104	jeff	jeff	
105	dave	passW0rD	

Acceso a múltiples registros no autorizados desde la interfaz web.

OffSec

Kali Linux

Kali Tools

Kali Docs

Kali Forums

Kali NetHunter

Exploit-DB

Google Hacking DB

Client side

Challenges

More specifically, if a user supplied string simply gets concatenated to a SQL query without any sanitization or preparation, then you may be able to modify the query's behavior by simply inserting quotation marks into an input field. For example, you could end the string parameter with quotation marks and input your own SQL after that.

It is your turn!

You are an employee named John **Smith** working for a big company. The company has an internal system that allows all employees to see their own internal data such as the department they work in and their salary.

The system requires the employees to use a unique *authentication TAN* to view their data. Your current TAN is **3SL99A**.

Since you always have the urge to be the most highly paid employee, you want to exploit the system so that instead of viewing your own internal data, *you want to take a look at the data of all your colleagues* to check their current salaries.

Use the form below and try to retrieve all employee data from the **employees** table. You should not need to know any specific names or TANs to get the information you need.

You already found out that the query performing your request looks like this:

```
"SELECT * FROM employees WHERE last_name = ' " + name + "' AND auth_tan = ' " +
```

✓

Employee Name:

Smith' or 1=1--

Authentication TAN:

3SL99A

Get department

You have succeeded! You successfully compromised the confidentiality of data by viewing internal information that you should not have access to. Well done!

USERID	FIRST_NAME	LAST_NAME	DEPARTMENT	SALARY	AUTH_TAN
32147	Paulina	Travers	Accounting	46000	P45JSI
34477	Abraham	Holman	Development	50000	UU2ALK
37648	John	Smith	Marketing	64350	3SL99A
89762	Tobi	Barnett	Development	77000	TA9LL1
96134	Bob	Franco	Marketing	83700	LO9S2V

Enumeración de tablas y estructuras internas mediante técnicas de inyección SQL.

Employee Name:
Authentication TAN:
Get department

You have succeeded! You successfully compromised the confidentiality of data by viewing internal information that you should not have access to. Well done!

USERID	FIRST_NAME	LAST_NAME	DEPARTMENT	SALARY	AUTH_TAN
null	ACCESS_CONTROL_USERS	null	null	null	null
null	ACCESS_LOG	null	null	null	null
null	ADMINISTRABLE_ROLE_AUTHORIZATIONS	null	null	null	null
null	APPLICABLE_ROLES	null	null	null	null
null	ASSERTIONS	null	null	null	null
null	ASSIGNMENT	null	null	null	null
null	ASSIGNMENT_PROGRESS	null	null	null	null
null	AUTHORIZATIONS	null	null	null	null
null	BLOCKS	null	null	null	null
null	CHALLENGE_USERS	null	null	null	null
null	CHARACTER_SETS	null	null	null	null
null	CHECK_CONSTRAINTS	null	null	null	null
null	CHECK_CONSTRAINT_ROUTINE_USAGE	null	null	null	null
null	COLLATIONS	null	null	null	null
null	COLUMNS	null	null	null	null
null	COLUMN_COLUMN_USAGE	null	null	null	null
null	COLUMN_DOMAIN_USAGE	null	null	null	null
null	COLUMN_PRIVILEGES	null	null	null	null
null	COLUMN_UDT_USAGE	null	null	null	null
null	CONSTRAINT_COLUMN_USAGE	null	null	null	null
null	CONSTRAINT_PERIOD_USAGE	null	null	null	null
null	CONSTRAINT_TABLE_USAGE	null	null	null	null
null	DATA_TYPE_PRIVILEGES	null	null	null	null
null	DOMAINS	null	null	null	null
null	DOMAIN_CONSTRAINTS	null	null	null	null
null	ELEMENT_TYPES	null	null	null	null
null	EMAIL	null	null	null	null
null	EMPLOYEES	null	null	null	null
null	ENABLED_ROLES	null	null	null	null
null	GRANT_RIGHTS	null	null	null	null
null	INFORMATION_SCHEMA_CATALOG_NAME	null	null	null	null

4.5 Impacto

La explotación de esta vulnerabilidad permite a un atacante acceder a información sensible que no debería estar disponible para usuarios no autorizados. En el entorno analizado, se obtuvo acceso a datos internos como nombres de usuario, contraseñas simuladas y cookies de sesión. En un entorno real, este tipo de vulnerabilidad podría permitir la exposición de credenciales o hashes de contraseñas, la suplantación de sesiones mediante cookies robadas y el acceso completo a cuentas de usuario, derivando en una brecha de seguridad de alto impacto.

4.6 Nivel de riesgo

Riesgo: ALTO

El impacto potencial es elevado debido a la exposición de información sensible, y la explotación puede realizarse de forma manual mediante la manipulación directa de los parámetros de entrada.

4.7 Recomendaciones de mitigación

- Implementar consultas preparadas.
- Validar y sanear todas las entradas de usuario.
- Aplicar el principio de mínimo privilegio en la base de datos.
- Evitar la exposición de información sensible en las respuestas.
- Realizar pruebas de seguridad periódicas durante el desarrollo.

5. Vulnerabilidad A3 – Cross-Site Scripting (XSS)

5.1 Descripción de la vulnerabilidad

El Cross-Site Scripting (XSS) es una vulnerabilidad que permite a un atacante inyectar código malicioso, normalmente JavaScript, en una aplicación web. Dicho código es posteriormente ejecutado en el navegador de otros usuarios que acceden a la aplicación vulnerable.

Este tipo de vulnerabilidad puede permitir el robo de cookies de sesión, la suplantación de identidad, la modificación del contenido de la página o la redirección a sitios maliciosos.

5.2 Vector de ataque identificado

Durante el análisis de la aplicación WebGoat se identificó un punto de entrada vulnerable dentro de la sección A3 Injection – Cross-Site Scripting, donde los datos introducidos por el usuario son reflejados en la respuesta de la aplicación sin aplicar un filtrado o codificación adecuados.

Este comportamiento permite que código JavaScript introducido por el usuario sea interpretado y ejecutado por el navegador.

5.3 Explotación de la vulnerabilidad

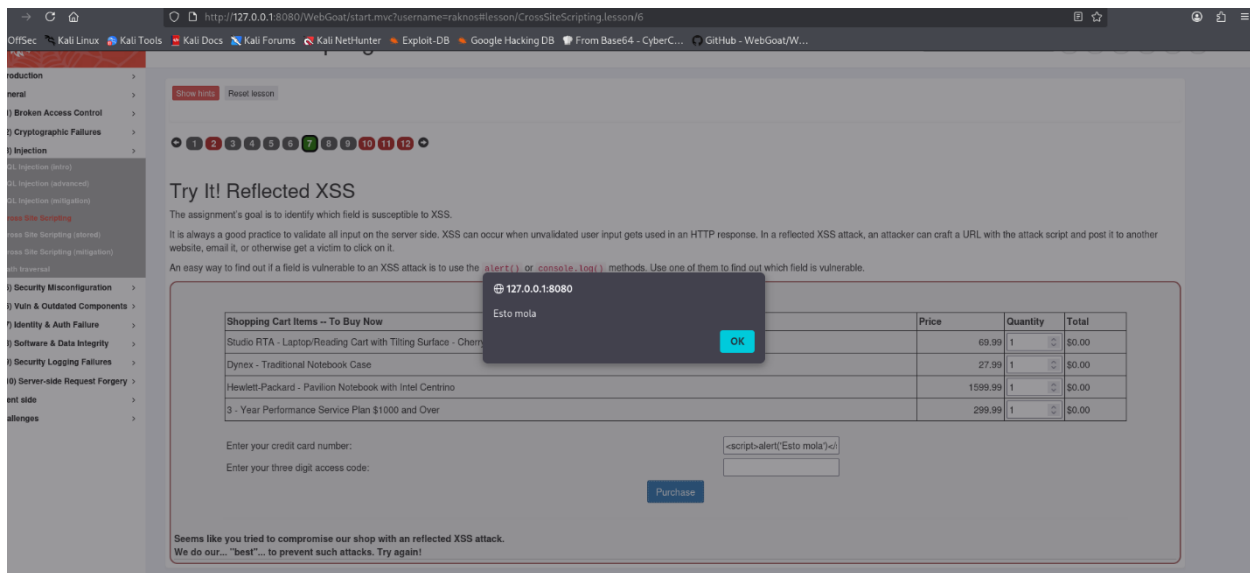
La explotación de la vulnerabilidad XSS se realizó de forma manual desde el navegador web, introduciendo código JavaScript en el campo de entrada vulnerable de la aplicación.

Al enviarse la petición, el contenido introducido fue reflejado directamente en la respuesta HTML sin ningún tipo de validación o escape, provocando la ejecución del código en el navegador.

Aunque en este ejercicio la explotación se ha realizado directamente desde el navegador, en un entorno real un ataque XSS reflejado podría llevarse a cabo mediante URLs manipuladas o enlaces maliciosos enviados a otros usuarios, sin necesidad de interacción directa con la aplicación por parte del atacante.

5.4 Evidencias

Ejecución de código JavaScript mediante Cross-Site Scripting



5.5 Impacto

La explotación de una vulnerabilidad XSS puede permitir a un atacante:

- Robar cookies de sesión.
- Suplantar la identidad de otros usuarios.
- Manipular el contenido de la aplicación.
- Redirigir a los usuarios a sitios maliciosos.

(En un entorno real, este tipo de vulnerabilidad supone un riesgo elevado para la seguridad de los usuarios y de la aplicación.)

5.6 Nivel de riesgo

Riesgo: MEDIO / ALTO

El impacto puede ser elevado, especialmente cuando se combina con fallos de gestión de sesión o autenticación.

5.7 Recomendaciones de mitigación

- Validar y sanear todas las entradas del usuario.
- Escapar correctamente la salida (output encoding) antes de mostrar datos introducidos por el usuario, de forma que los caracteres especiales sean tratados como texto y no como código ejecutable por el navegador.
- Implementar políticas de seguridad como **Content-Security-Policy (CSP)**.

- Evitar reflejar directamente entradas del usuario en la respuesta HTML.

6. Vulnerabilidad A5 – Security Misconfiguration

6.1 Descripción de la vulnerabilidad

La Security Misconfiguration hace referencia a configuraciones inseguras o incorrectas en la aplicación, el servidor o la infraestructura que pueden ser aprovechadas por un atacante. Este tipo de vulnerabilidad suele aparecer cuando se utilizan configuraciones por defecto, servicios innecesarios habilitados, mensajes de error excesivamente detallados o cabeceras de seguridad ausentes.

Estas configuraciones débiles amplían la superficie de ataque y permiten a un atacante recolectar información clave para comprometer otras capas del sistema.

6.2 Vector de ataque identificado

Durante el análisis de la aplicación WebGoat se identificaron diversas configuraciones de seguridad mejorables que podrían facilitar ataques o la exposición de información interna.

6.3 Explotación / Identificación de la vulnerabilidad

La identificación de esta vulnerabilidad se realizó mediante **análisis manual**, accediendo a la aplicación desde el navegador y observando su comportamiento y configuración visible.

Este análisis evidenció elementos como la exposición de información interna, configuraciones por defecto y ausencia de determinadas medidas de seguridad habituales en aplicaciones web modernas.

Este acceso no autorizado a datos sensibles representa una puerta de entrada significativa en un escenario real. Aunque en esta auditoría no se realizó una fase completa de post-explotación, debe considerarse que las credenciales y tokens obtenidos podrían ser utilizados para escalar privilegios, comprometer otros sistemas internos o mantener persistencia en el entorno.

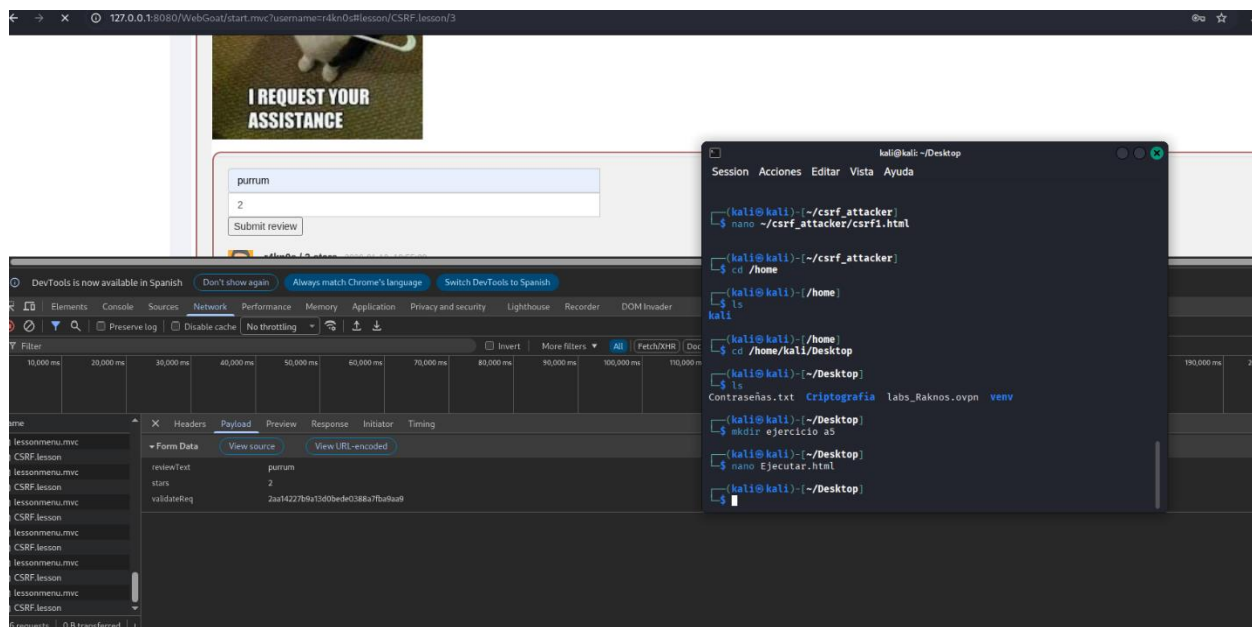
Este tipo de vectores suele aprovecharse en entornos reales para realizar movimientos laterales o acceder a recursos más críticos, lo que refuerza la criticidad de la vulnerabilidad explotada.

6.4 Evidencias

La identificación de esta vulnerabilidad se realizó principalmente mediante análisis manual desde el navegador, utilizando las herramientas de desarrollo y la ejecución de contenido HTML de prueba para observar el comportamiento de la aplicación y sus configuraciones de seguridad.

En paralelo, este tipo de análisis puede apoyarse con herramientas especializadas de análisis de tráfico HTTP, las cuales permiten inspeccionar las respuestas del servidor, cabeceras HTTP y cookies de sesión. Ambas aproximaciones permiten identificar configuraciones inseguras sin necesidad de realizar una explotación activa de la aplicación.

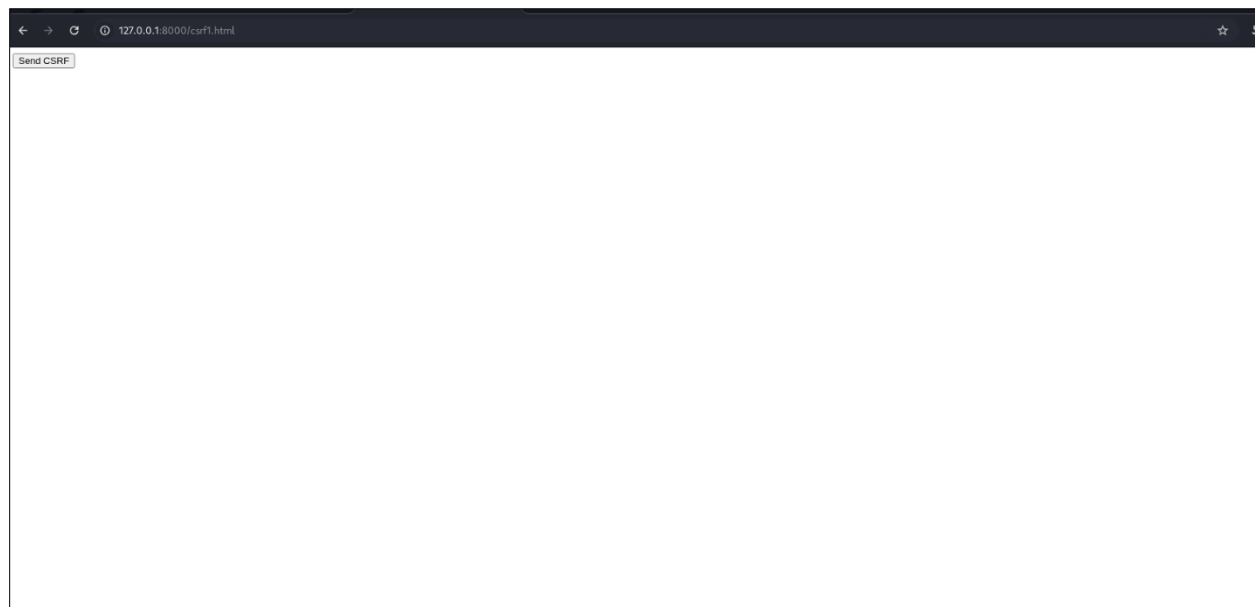
Preparación del ataque CSRF mediante archivo HTML externo



El código con HTML con el que vamos a lanzar al navegador

```
(kali@kali)-[~/Desktop]
$ cat Ejecutar.html
<html>
<form action=http://127.0.0.1:8080/WebGoat/csrf/review method=post enctype='application/x-www-form-urlencoded; charset=UTF-8'>
  <input name='reviewText' value='Great' type='hidden'>
  <input name='stars' value='5' type='hidden'>
  <input name='validateReq' value='2aa14227b9a13d0bede0388a7fba9aa9' type='hidden'>
  <input type='submit' value='submit'>
</form>
</html>
(kali@kali)-[~/Desktop]
```

Página objetivo antes de la ejecución del ataque CSRF



6.5 Impacto

Las configuraciones de seguridad incorrectas pueden permitir a un atacante obtener información sensible sobre la aplicación, el servidor o el entorno de ejecución, facilitando ataques más complejos como inyección, escalado de privilegios o ataques a la autenticación. (En un entorno real, este tipo de vulnerabilidad puede aumentar significativamente la superficie de ataque de la aplicación.)

6.6 Nivel de riesgo

Riesgo: MEDIO

El impacto depende del tipo de información expuesta y de la combinación con otras vulnerabilidades presentes en la aplicación.

6.7 Recomendaciones de mitigación

- Revisar y endurecer las configuraciones de seguridad del servidor y de la aplicación.
- Deshabilitar servicios y funcionalidades innecesarias.
- Evitar la exposición de mensajes de error detallados en entornos productivos.
- Implementar cabeceras de seguridad HTTP adecuadas.
- Seguir guías de hardening y buenas prácticas de despliegue seguro.

7. Vulnerabilidad A6 – Vulnerable & Outdated Components

7.1 Descripción de la vulnerabilidad

La categoría Vulnerable and Outdated Components hace referencia al uso de componentes, librerías o frameworks de terceros que se encuentran desactualizados o presentan vulnerabilidades conocidas. Este tipo de vulnerabilidad puede existir incluso cuando el código desarrollado por la aplicación es correcto, ya que el fallo reside en dependencias externas. Tal y como indica OWASP, una aplicación puede ser vulnerable debido a componentes inseguros, aun cuando el desarrollo propio no contenga errores de programación.

7.2 Identificación en el entorno analizado

Durante el análisis de la aplicación WebGoat se identificó un ejemplo claro de esta vulnerabilidad mediante el uso de distintas versiones de un mismo componente de terceros, concretamente la librería **jquery-ui**.

Utilizando el mismo código fuente de la aplicación, se observó un comportamiento diferente en función de la versión del componente empleado:

- **jquery-ui versión 1.10.4:** versión vulnerable que permite la explotación de una vulnerabilidad de tipo Cross-Site Scripting (XSS).
- **jquery-ui versión 1.12.0:** versión actualizada y no vulnerable, en la cual la explotación no es posible.

La evidencia obtenida sugiere que la vulnerabilidad se introduce a través de un componente de terceros no actualizado, descartando problemas en la lógica interna de la aplicación

7.3 Explotación / Evidencia de la vulnerabilidad

En el caso de la versión vulnerable de jquery-ui (1.10.4), la aplicación permite que el contenido del parámetro **closeText** sea controlado por el usuario sin aplicar los mecanismos de protección adecuados, lo que posibilita la ejecución de código JavaScript en el navegador.

Al actualizar la librería a una versión no vulnerable (1.12.0), el mismo comportamiento deja de producirse, eliminando por completo la posibilidad de explotación, aun manteniendo el mismo código de la aplicación.

Este caso ilustra claramente cómo una dependencia no actualizada puede comprometer la seguridad de la aplicación.

7.4 Impacto

El uso de componentes vulnerables o desactualizados puede permitir a un atacante explotar vulnerabilidades conocidas sin necesidad de descubrir nuevos fallos en el código de la aplicación. En un entorno real, este tipo de vulnerabilidad puede facilitar ataques como la ejecución de código en el navegador del usuario, el robo de información sensible o la suplantación de identidad.

Además, el uso de versiones antiguas reduce significativamente el esfuerzo necesario para llevar a cabo ataques, ya que las vulnerabilidades suelen estar ampliamente documentadas y automatizadas.

7.5 Nivel de riesgo

Riesgo: MEDIO / ALTO

El nivel de riesgo depende del tipo de componente vulnerable y de la criticidad de la vulnerabilidad asociada. En este caso, al permitir la ejecución de código JavaScript en el navegador del usuario, el impacto puede considerarse elevado en un entorno productivo.

7.6 Recomendaciones de mitigación

- Mantener actualizadas todas las librerías, frameworks y componentes de terceros utilizados por la aplicación.
- Monitorizar vulnerabilidades conocidas asociadas a las dependencias (CVE, avisos de seguridad).
- Eliminar componentes obsoletos o innecesarios.
- Implementar un proceso de gestión de dependencias y actualizaciones periódicas.

- Evitar permitir que componentes de terceros procesen directamente entradas controladas por el usuario sin validación adecuada.

8. Vulnerabilidad A7 – Identity & Authentication Failure (Secure Passwords)

8.1 Descripción de la vulnerabilidad

La vulnerabilidad Identity & Authentication Failure se produce cuando los mecanismos de autenticación no están correctamente diseñados o implementados. Uno de los factores más comunes es el uso de contraseñas débiles, fácilmente adivinables o vulnerables a ataques de fuerza bruta y diccionario.

Una gestión inadecuada de las contraseñas puede permitir a un atacante comprometer cuentas de usuario, acceder a información sensible y realizar acciones no autorizadas dentro de la aplicación.

8.2 Identificación en el entorno analizado

Durante el análisis del apartado **Secure Passwords** de la aplicación **WebGoat**, se pone de manifiesto la importancia de utilizar contraseñas robustas y difíciles de adivinar.

El ejercicio demuestra cómo la fortaleza de una contraseña influye directamente en el tiempo necesario para ser descubierta mediante ataques de fuerza bruta. WebGoat evalúa la calidad de la contraseña introducida y requiere un nivel mínimo de seguridad para completar el ejercicio correctamente.

8.3 Análisis del problema

El análisis del apartado Secure Passwords evidencia que la resistencia de una contraseña frente a ataques de fuerza bruta depende directamente de su longitud y de la diversidad de los caracteres utilizados en su composición.

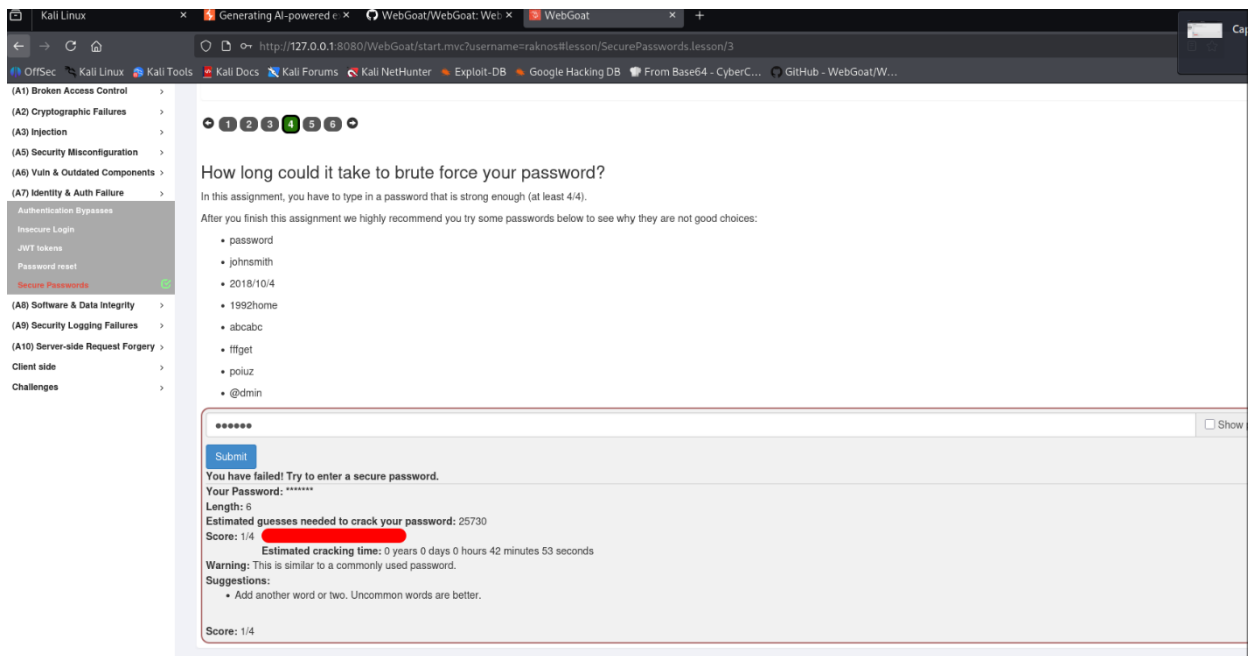
Contraseñas con una longitud reducida o basadas en patrones simples disminuyen significativamente el número de combinaciones posibles, lo que facilita su descubrimiento mediante herramientas automatizadas en un periodo de tiempo reducido. Este tipo de contraseñas no ofrece un nivel de protección adecuado frente a ataques de fuerza bruta o diccionario.

Por el contrario, el uso de contraseñas más largas y que combinan letras mayúsculas, minúsculas, números y caracteres especiales incrementa exponencialmente el espacio de búsqueda, haciendo que el tiempo necesario para comprometerlas sea inviable en un contexto realista.

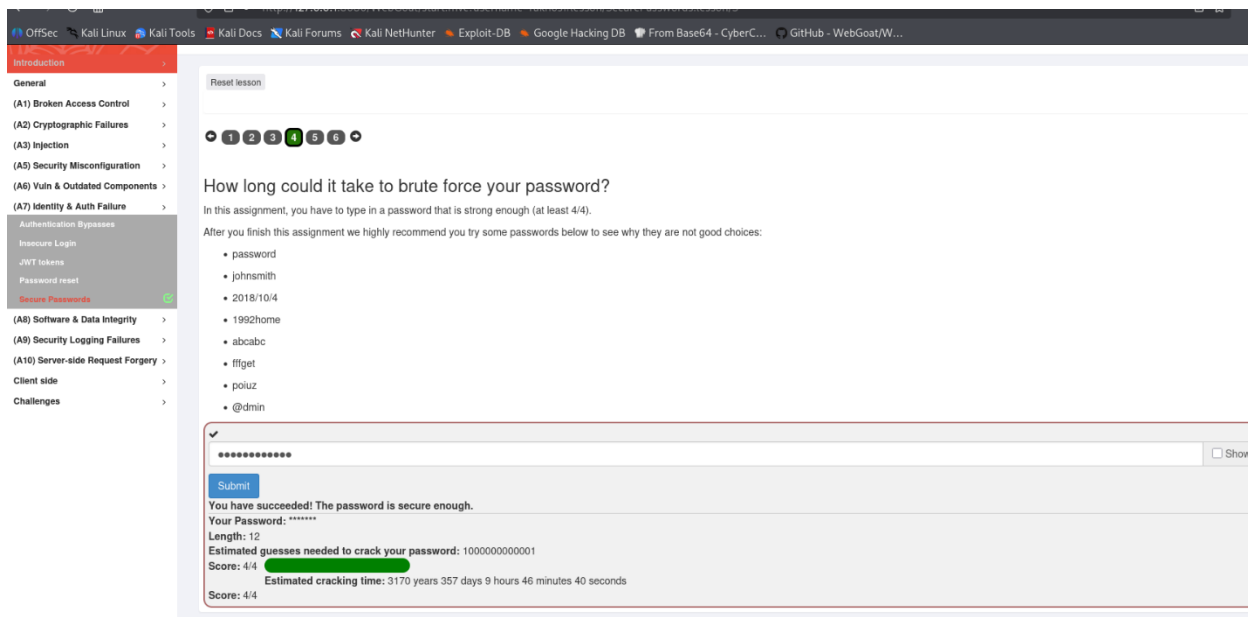
Este comportamiento pone de manifiesto la necesidad de aplicar políticas de contraseñas robustas como parte fundamental de los mecanismos de autenticación, reduciendo significativamente el riesgo de compromiso de cuentas de usuario.

8.4 Evidencias

Evaluación de una contraseña débil, cuya baja complejidad permite su ruptura en un periodo de tiempo reducido mediante ataques de fuerza bruta



Evaluación de una contraseña robusta que, debido a su longitud y complejidad, requiere un tiempo de ruptura inviable mediante fuerza bruta



Las evidencias muestran de forma clara cómo la longitud y complejidad de una contraseña influyen directamente en el tiempo necesario para su compromiso, reforzando la necesidad de aplicar políticas de contraseñas robustas.

8.5 Impacto

Una gestión insegura de contraseñas puede permitir a un atacante acceder a cuentas de usuario sin autorización, comprometiendo la confidencialidad de la información y permitiendo acciones maliciosas en nombre de otros usuarios.

En un entorno real, este tipo de vulnerabilidad puede derivar en brechas de seguridad graves, especialmente si las credenciales comprometidas se reutilizan en otros servicios.

8.6 Nivel de riesgo

Riesgo: MUY ALTO

El impacto asociado a fallos en la autenticación es elevado, ya que afecta directamente a la identidad de los usuarios y al control de acceso a la aplicación.

8.7 Recomendaciones de mitigación

- Aplicar políticas de contraseñas robustas (longitud mínima, complejidad).
- Almacenar las contraseñas utilizando algoritmos de hash seguros y con *salt*.
- Evitar el almacenamiento de contraseñas en texto plano.
- Implementar mecanismos de protección frente a ataques de fuerza bruta.
- Fomentar el uso de autenticación multifactor cuando sea posible.
- Usar aplicaciones externas de gestión de contraseñas

En conclusión, esta auditoría no solo permitió identificar vulnerabilidades críticas dentro de un entorno controlado, sino que también demostró la efectividad de una metodología estructurada basada en estándares reconocidos. La experiencia adquirida a través de este ejercicio refuerza la necesidad de mantener una cultura de seguridad proactiva, donde la evaluación continua y el aprendizaje práctico se integren como pilares esenciales del desarrollo seguro.